Evaluating Quantum Circuits Across Theoretical Fault-Tolerant Architecture Models

PHYS 765 FINAL PROJECT – SPRING 2026

ADI RAVI & MAX BUBLIK
UNIVERSITY OF WISCONSIN - MADISON |

Introduction and Motivation

Quantum circuits are often analysed long before the machines that may eventually execute them exist. At the circuit level, it is natural to compare algorithms using logical measures such as gate count and circuit depth. A fault-tolerant quantum computer, however, will not execute an abstract circuit diagram directly. It will expose a particular logical layout, supported operations, connectivity structure, timing model, and communication cost between different parts of the system. These architectural choices can significantly change whether a circuit that appears efficient in an abstract model remains efficient after compilation. [7], [8], [9].

Much of today's quantum software development still takes place in the noisy intermediate-scale quantum (NISQ) regime, where sparse connectivity, native gate sets, noise, and limited coherence strongly shape execution. Preskill's NISQ framework highlights the importance of these near-term noisy devices while also emphasizing that scalable quantum computation will ultimately require improved gate fidelities and fault tolerance. Future fault-tolerant systems are likely to introduce a different set of constraints, including logical qubits, tiled layouts, inter-block communication links, and logical operations with architecture-dependent costs. This creates a gap between circuit design and architecture dependent evaluation: a circuit may be specified abstractly, but its practical usefulness depends on how it maps to a target architecture. [1], [7], [11]

Existing quantum software frameworks provide strong support for compiling circuits to present-day hardware and custom backend models. However, there is less support for rapidly defining hypothetical fault-tolerant-style architectures directly from user-provided assumptions and evaluating circuits against them as compiler targets. Our project addresses this need by using Qiskit's target abstraction in a different way: instead of describing only an existing machine, we construct a custom target from a user-defined fault-tolerant-style architecture profile. [3], [5], [6]

Given a circuit and an architecture profile, specified through a JSON or dictionary-based description, the framework builds a `Target` containing the chosen topology, supported gates, modeled gate errors, and modeled gate durations. The circuit can then be transpiled against this target and evaluated using model-based architectural metrics, including communication overhead, scheduled duration estimates, and first-order success proxies.

The goal of this tool is not to simulate a complete fault-tolerant execution stack, surface-code decoder, or calibrated physical hardware system. Instead, the project focuses on the logical architecture and compilation layer: how circuit structure changes when the compiler is given different fault-tolerant-style architectural assumptions. In this sense, the framework serves as an early-stage bridge between abstract quantum circuit design, architecture dependent transpilation, and future full-stack quantum resource estimation.

Tool Framework (Pipeline) [3], [4], [5], [6]

1. Assumptions

- The architecture is modeled at the logical target level, not at the full physical error-correction level.
- The input circuit is assumed to have no inherent noise that can propagate through mapping and transpilation.
- User-specified profiles provide the relevant machine description through topology plus per-gate modeled error and duration values.
- Gate failures are treated through a first-order independent-error model when computing success-style metrics.
- Compilation is performed through Qiskit's transpilation infrastructure, so the reported behavior depends on the chosen compiler settings, especially the preset Level 3 transpiler.
- Connectivity assumptions are imposed through abstract layouts such as **tiled_k_nearest**, **heavy_hex**, and **heavy_square** rather than derived from a calibrated hardware control stack.
- The framework assumes that comparing compiled depth, routing overhead, modeled duration, and success proxy is a meaningful early-stage way to compare architectures.

2. Circuit Input

The first stage of the framework is circuit ingestion and normalization. At this stage, the circuit is treated as a logical program rather than as instructions tied to any one hardware backend. The purpose of this layer is to accept a user-specified circuit, convert it into a standard internal representation, and preserve structural information such as qubit count, gate content, and circuit depth before architecture dependent compilation begins.

In the current MVP, the repository supports several input paths rather than a single unified frontend. Built-in benchmark and test circuits can be created directly in code through utility modules, while externally defined circuits can be introduced through OpenQASM-compatible representations. This reflects the broader design goal of allowing the tool to evaluate circuits independently of the original circuit authoring environment.

The current input flow is structured as follows:

1. **Circuit definition or loading.**
   - Built-in circuits are created in *circuit_loader.py (line 1)*. In the current MVP, this file includes directly defined example circuits, including a small PennyLane test circuit as well as benchmark-style Qiskit circuits such as a Toffoli cascade and a trotterized spin-chain circuit. At this stage, the circuit remains a logical

object specified by its algorithmic structure rather than by any particular hardware backend.

2. **QASM export or QASM ingestion.**
   - If the circuit originates in PennyLane, *export_qasm.py (line 1)* can be used to convert it into an OpenQASM string. If the user already has a circuit written in OpenQASM form, *qasm_ingestor.py (line 1)* can ingest either a QASM string or a QASM file directly. This step reduces dependence on the original circuit frontend and gives the pipeline a more standardized circuit description.
3. **Conversion to the internal representation.**
   - Once a valid QASM description is available, *qasm_to_ir.py (line 1)* converts it into a Qiskit *QuantumCircuit*. This *QuantumCircuit* serves as the effective intermediate representation for the rest of the framework, since the downstream **Target** construction, transpilation, and metric evaluation stages are built around Qiskit's compiler infrastructure.

3. Architecture Model and Target Creation

The architecture profile is where the hardware assumptions enter the framework. It defines the logical topology of the target machine, the gates it supports, and modeled operation properties such as error and duration. These assumptions matter because the same logical circuit can compile differently depending on connectivity, native gate support, and whether interactions are local or require communication across blocks.

Rather than simulating physical error-correction cycles, the framework models fault-tolerant-style hardware at the logical architecture level. Currently, the architecture layer is built around user-defined profiles instead of fixed physical backends. A user provides a configuration describing the layout and operation properties of the target architecture, and the tool converts that profile into a custom Qiskit **Target**. This allows the same input circuit to be transpiled and evaluated under different architecture assumptions.

1. **Architecture profile specification.**
   - A target architecture is defined through a configuration consisting of a topology block and a profile block. The topology block describes the machine layout, while the profile block specifies the supported gates together with modelled error rates and durations. This configuration is handled in *target.py (line 19).* A template for this type of user-defined input is also provided in *blueprint.json (line 1).* So, the hardware model is supplied by the user as an abstract architecture profile rather than being tied to a single physical device.
2. **Connectivity map generation.**
   - Once the configuration is supplied, *target.py (line 33)* selects the requested topology type and delegates connectivity construction to *connectivity.py (line 80)*. For modular logical layouts, **k_nearest_tiled_coupling_map** generates block-

structured coupling maps with tunable intra-block and inter-block connectivity. For more code inspired layouts, generate_modular_layout constructs tiled heavy-hex and heavy-square connectivity graphs in *connectivity.py (line 346)*. The repository also includes CSV-based IBM connectivity loaders for devices such as Fez and Torino, allowing the framework to reference more hardware-like sparse graphs when needed, and the ability to load a custom connectivity through user defined CSV files.

3. **Target construction and gate assignment.**
    - After the connectivity map is created, *target.py* validates the gate profile, converts user-specified gate names into Qiskit gate objects, and populates a custom Qiskit **Target** with per-qubit and per-edge **InstructionProperties** . This logic is implemented by the **FTarget** class, while *FaultTolerantTarget.py* provides a simpler user-facing wrapper. Single-qubit gates are assigned to each qubit, while two-qubit and inter-device operations follow the generated coupling map. This allows the framework to distinguish local intra-block operations from more expensive inter-block communication while giving the transpiler a machine model that includes both topology and modelled gate costs.

4. Compilation and transpilation

Once the circuit has been normalized into a Qiskit QuantumCircuit object and the architecture profile has been converted into a custom Qiskit **Target**, the framework proceeds to compilation and transpilation. Because the resulting machine model is represented as a standard Qiskit **Target**, it can in principle be used with any compatible Qiskit compiler workflow, including preset transpilation pipelines, custom pass managers, and other target-aware tooling. In the current MVP, the active examples primarily use Qiskit's preset transpiler, while the repository also defines a modular internal pass-manager structure that allows the major compilation stages to be explicitly customized.

The current compilation flow is structured as follows:

1. **Preset Qiskit transpilation.**
    - In the active example workflow, the circuit is compiled using Qiskit's preset pass manager, generate_preset_pass_manager, at *optimization level 3*, in *basic_distance_eval.py (line 68).* This allows the framework to compile directly against the custom Target without requiring a fully custom transpiler stack for every experiment. This way, the project uses Qiskit's existing compiler infrastructure while still allowing user-defined fault-tolerant-style architectures to serve as compilation targets.

2. **Modular pass-manager framework.**

Although the current examples primarily use Qiskit's preset transpiler, the repository also includes a modular custom pass-manager structure in the *backend/pass_managers* directory for future customization of the transpilation workflow.

- **Initialization**.

initializer.py (line 7) defines **get_init_pm**, which applies HighLevelSynthesis and can optionally unroll custom definitions into lower-level circuit instructions. This stage acts as a preprocessing layer before architecture-specific mapping begins.

- **Layout and routing.**

layout_n_routing.py (lines 9, 33, and 55) defines **get_layout_pm**, **get_routing_pm**, and **get_layout_routing_pm**. These functions use SABRE-based heuristics to assign logical qubits to physical qubits and to insert SWAP operations when two-qubit gates cannot be executed directly under the target connectivity map. This stage is especially important for modular or sparse architectures, where communication overhead can strongly affect the final compiled circuit.

- **Translation.**

translator.py (line 27) defines **get_translation_pm**, which uses Qiskit's **BasisTranslator** together with single-qubit gate decomposition cleanup. This stage forces the circuit into the native instruction set architecture supported by the chosen target. In the current codebase, this can include more conventional superconducting-style bases as well as more abstract fault-tolerant-style logical bases.

- **Optimization.**

optimizer.py (line 14) defines **get_optimization_pm**, which currently applies lightweight circuit simplifications such as commutative cancellation and reset removal. The role of this stage is to reduce redundant structure without introducing a more expensive synthesis-heavy optimization flow.

- **Scheduling.**

scheduling.py (line 4) defines **get_scheduling_pm**, which applies ALAP scheduling and inserts delay padding when gate-duration information is available through the target. This stage is only meaningful when the architecture profile includes modeled timing data, since scheduling depends on the duration information attached to the custom Target.

3. **Compiled circuit generation.**

- After these compilation stages are applied, the output is a transpiled circuit whose structure reflects the constraints of the selected architecture. At this point, the circuit has been rewritten in terms of the target's supported operations, mapped onto its connectivity graph, and modified to include any routing overhead introduced by the architecture. This compiled circuit then becomes the input to the metric-evaluation stage.

Results and Validation

To demonstrate the framework, we focus on the worked example presented in *01_Target_Creation.ipynb*, where a user-defined fault-tolerant-style architecture profile is converted into a custom Qiskit **Target** through the **FTTarget** interface. This example serves as a concrete validation of the overall workflow: rather than reasoning only in abstract terms about compiler targets, it shows that a user can specify an architecture profile, instantiate the corresponding target object, and use that target as the basis for architecture dependent compilation and evaluation. The demonstration uses a standard Qiskit Circuit library with parameters of duration and error determined by mathematical models of logical surface code error scaling by a determined threshold for a surface code and a general physical gate error [11].

Below is the figure generated when circuit depth is iteratively increased on a singular device with physical error rate below the error threshold of the surface code, done on a 100-qubit random circuit with initial depth 10. The overall circuit error probability is taken as the product of every gate probability per instance of said gate as defined by our target object

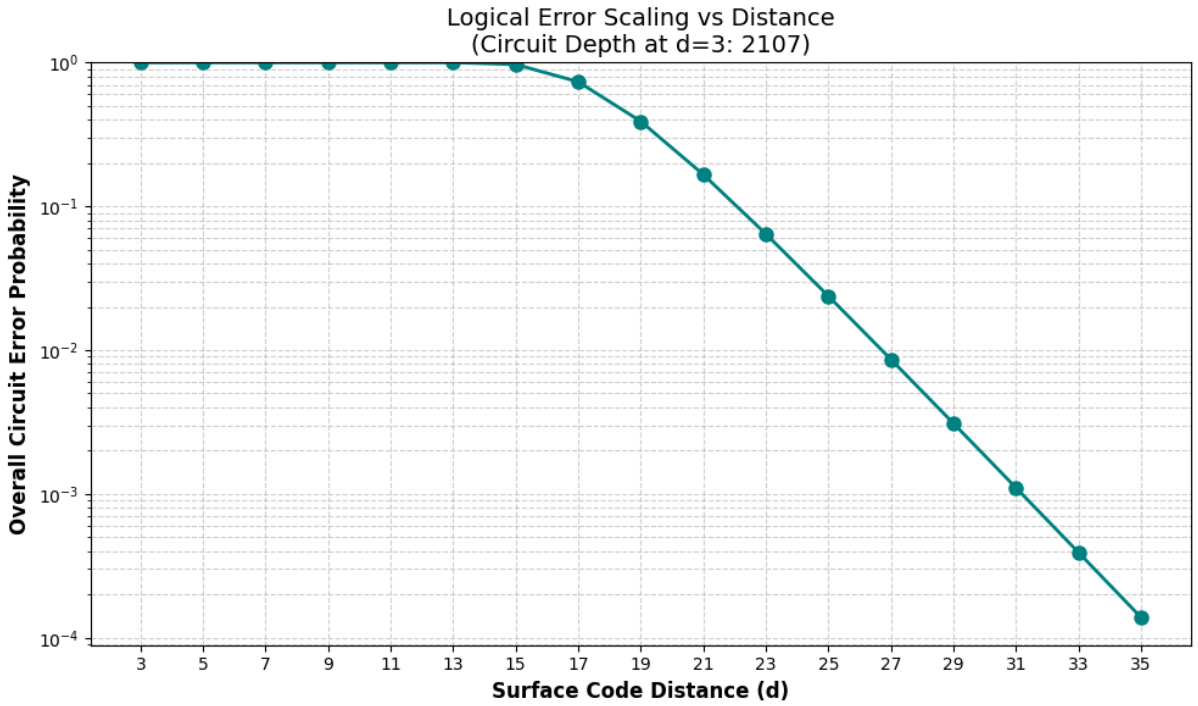

Figure 1

The total circuit probability reduces at an exponential rate once it reaches a depth of about 15, showing how increasing the depth of a surface code allows for arbitrarily high fidelity by a simple increase in depth count.

Contrarily, by transpiling the same circuit onto a networked connectivity of 4 devices with a networked error rate of 0.5%, the error rate converges to a total circuit error probability of 99.765455%.

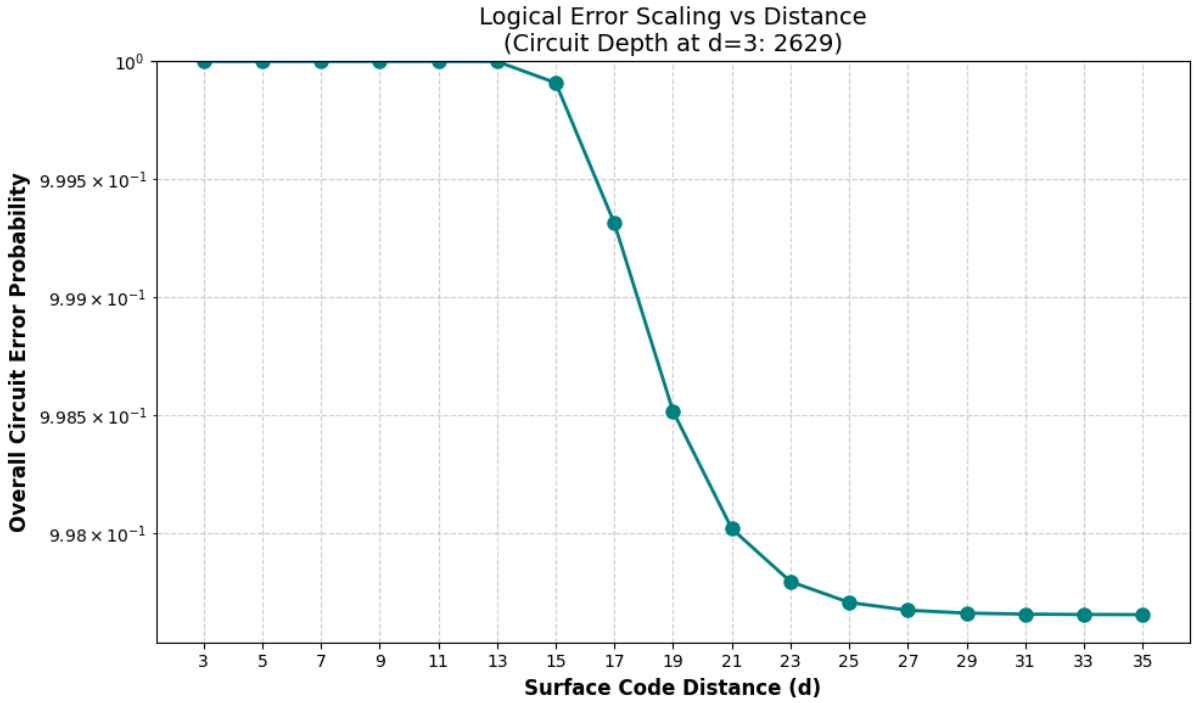

Figure 2

Notably, if we take

$$1 - (1 - error\_of\_photonic\_connection)^{number\_of\_photonic\_gates},$$

we find the convergent limit of the overall success of the circuit at 99.765425%., as shown in the demo.

This workflow is one example of a use-case for our networked FTTarget , showing how arbitrarily low logical error rates are not enough to construct a fully networked Fault Tolerant device.

Full Stack Implications

Fault-tolerant quantum computers should not be evaluated only by the number of logical qubits they provide. Their performance will also depend on how those qubits are arranged,

which operations are available locally, and what cost is incurred when information must move across different regions or modules of the machine. An architecture may appear scalable in qubit count but still introduce large overheads if useful circuits require frequent long-range interactions. [7], [9], [10]

This is where the project fits into the full stack. Many fault-tolerant architecture proposals are described in terms of layouts, modules, communication links, or logical operations. However, those descriptions do not immediately show how a given circuit changes after compilation. The resource question is therefore not only how many gates a circuit contains, but how well the circuit's interaction pattern matches the architecture intended to support it. [7], [8]

At the **circuit and algorithm level**, the framework shifts attention from the abstract gate list to the circuit's interaction structure. Two circuits with similar gate counts or depths can place very different demands on a target architecture. A circuit whose two-qubit gates are mostly local under one layout may require additional routing under another. For fault-tolerant-style architectures, this matters because routing and communication can become part of the resource cost rather than a minor implementation detail. The framework gives circuit designers a way to test whether an algorithm's logical structure is compatible with a proposed architecture model before moving to a more detailed physical or error-correction analysis. [8]

At the **logical architecture layer**, the project treats fault-tolerant layouts as models that can be tested through compilation. A proposed architecture may specify blocks, tiles, coupling regions, or communication links, but its practical value depends on how those structures support actual circuit interactions. Increasing connectivity within a block may reduce local routing overhead, while increasing connectivity between blocks may reduce expensive communication steps. By making these assumptions explicit in architecture profiles, the framework allows different fault-tolerant-style layouts to be compared in terms of how they shape the compiled circuit, rather than only how they look structurally. [7], [9], [10]

The **compiler laye**r is where architecture-circuit mismatch becomes visible. If a circuit assumes interactions that the target architecture does not directly support, the compiler must resolve that mismatch through routing, basis translation, or scheduling choices. These changes are not just software details; they determine the circuit that is evaluated under the architecture model. In this sense, transpilation is the point where abstract circuit design meets architectural constraint. By making the target architecture explicit, the framework helps identify whether overhead comes from the algorithm itself, the chosen layout, the supported gate set, or the communication structure. [10], [8]

At the **resource-estimation layer**, the framework provides early indicators of where architectural cost enters the compiled circuit. These metrics should not be read as final hardware predictions, but as model-based signals. A high inter-block operation count suggests that the circuit places heavy demand on communication between regions of the architecture. A longer scheduled duration estimate suggests that timing assumptions and available parallelism are limiting execution. Unsupported-operation counts show where the

chosen gate set is incomplete for the circuit being tested. The independent-error success proxy gives a first-order way to compare profiles under the same assumptions, while still leaving out correlated errors, decoder behaviours, leakage, and physical error-correction dynamics. [8], [9], [10]

The broader implication is that this framework can act as a bridge between abstract circuit design and future fault-tolerant resource estimation. It does not replace detailed surface-code compilers, decoder simulations, or physical hardware models. Instead, it provides a configurable first layer where users can test how architecture choices affect compiled circuit behaviours. As the tool matures, this same target-based approach could be extended toward more realistic logical-to-code mappings, deeper use of scheduling, and calibrated architecture profiles. [7], [8], [9]

Limitations and future work

Some limitations are:

- The tool does not simulate a full fault-tolerant execution stack, decoder, lattice surgery schedule, or physical surface-code cycle-by-cycle process.
- The reported success metric is only a proxy; it does not capture correlated noise, crosstalk, leakage, drift, decoder failure, or time-dependent hardware effects.
- Duration estimates depend on modeled gate durations attached to the custom Target; they are not calibrated runtime predictions for a real machine.
- The custom pass-manager framework exists, but the current working examples primarily rely on Qiskit's preset transpiler rather than a fully integrated custom compilation stack.
- The heavy-hex and heavy-square paths are less robust than the tiled workflow because they depend on additional layout tooling.
- Input handling is not yet a polished end-user interface; the repo supports multiple paths, but they are not fully unified into one clean frontend workflow.
- Validation is currently compiler and architecture level, not experimental hardware validation.

Despite its current limitations, the framework provides a strong foundation for several meaningful extensions. The present version already shows that user-defined architecture profiles can be converted into custom Qiskit **Targets** and used for architecture dependent compilation. Future work can build on this core capability by improving the input interface,

strengthening the fault-tolerant mapping model, and incorporating more realistic performance evaluation tools.

A natural next step is to build a more user-friendly CLI or GUI around the current framework. The backend already supports custom targets, user-defined profiles, and architecture dependent compilation, but these capabilities are still exposed mainly through code-level workflows. A dedicated interface would simplify navigation, profile selection, circuit loading, and experiment execution, making the tool more accessible and more practical for repeated use.

A more substantial extension is to model fault tolerance more explicitly beyond the current logical connectivity abstraction. In the present framework, the fault-tolerant target is treated as an already error-corrected architecture with prescribed logical operations, durations, and error rates. A natural next step would be to map compiled logical circuits more directly onto surface-code-style structures, including explicit treatment of lattice surgery or related logical operation models. In that setting, the framework would move closer to an open-source backend for compiling, optimizing, and mapping logical circuits onto a more realistic fault-tolerant substrate. The framework to create `tiled heavy_hex` and `heavy_square` Qiskit circuit objects exists as a proof of concept for how this compilation could occur within the framework of `FTTarget`, allowing for an all-in-one logical transpilation with mapping to surface code layout. The logical transpilation could be modified to accept arguments specifically geared around the relative cost of each gate within a fault tolerant code, or this could be inscribed within the error parameter as many transpilers are centred around minimizing the error. [7]

Finally, the inclusion of explicit noise simulation through `Qiskit Aer` would make the evaluation layer more realistic. The current metrics are intentionally lightweight and model based, but integrating Aer-level noise models would allow circuits to be analysed under more detailed assumptions about gate noise and execution behaviours. Combined with more explicit surface-code-inspired mappings, this would strengthen the connection between the current architecture dependent benchmarking framework and future full-stack fault-tolerant resource estimation. [12]

Contribution Split

This project was completed collaboratively by Adi and Max, with both partners contributing equally to the final outcome. Max led a larger share of the core software implementation including tiling logic, target generation/validation, and constructing the demo. Adi contributed to project design, implementation support, scientific framing, documentation, and report writing.

References & Appendix

[1] J. Preskill, “Quantum Computing in the NISQ era and beyond,” *Quantum*, vol. 2, p. 79, 2018. Available: https://quantum-journal.org/papers/q-2018-08-06-79/

[2] IBM Quantum, “IBM lays out clear path to fault-tolerant quantum computing.” Available: https://www.ibm.com/quantum/blog/large-scale-ftqc

[3] IBM Quantum Documentation, “Qiskit transpiler Target API documentation.” Available: https://quantum.cloud.ibm.com/docs/api/qiskit/qiskit.transpiler.Target

[4] IBM Quantum Documentation, “Qiskit transpiler InstructionProperties API documentation.” Available: https://quantum.cloud.ibm.com/docs/api/qiskit/qiskit.transpiler.InstructionProperties

[5] IBM Quantum Documentation, “Qiskit transpiler API documentation.” Available: https://quantum.cloud.ibm.com/docs/api/qiskit/transpiler

[6] IBM Quantum Documentation, “Introduction to Qiskit.” Available: https://quantum.cloud.ibm.com/docs/guides/tools-intro

[7] D. Litinski, “A Game of Surface Codes: Large-Scale Quantum Computing with Lattice Surgery,” *Quantum*, vol. 3, p. 128, 2019. Available: https://quantum-journal.org/papers/q-2019-03-05-128/

[8] T. LeBlond et al., “Realistic Cost to Execute Practical Quantum Circuits using Direct Clifford+T Lattice Surgery Compilation,” arXiv:2311.10686, 2023. Available: https://arxiv.org/abs/2311.10686

[9] S. N. Saadatmand et al., “Fault-tolerant resource estimation using graph-state compilation on a modular superconducting architecture,” arXiv:2406.06015, 2024. Available: https://arxiv.org/abs/2406.06015

[10] A. Ovide, S. Rodrigo, M. Bandic, H. van Someren, S. Feld, S. Abadal, E. Alarcón, and C. G. Almudéver, “Mapping quantum algorithms to multi-core quantum computing architectures,” arXiv:2303.16125, 2023. Available: https://arxiv.org/abs/2303.16125

[11] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, “Surface codes: Towards practical large-scale quantum computation,” arXiv:1208.0928, 2012. Available: https://arxiv.org/abs/1208.0928

[12] IBM Quantum Documentation, “Qiskit Aer.” Available: https://qiskit.github.io/qiskit-aer/